

NEURONAudit

Automated static analysis of NEURON repositories

Dominik Suwała, Mateusz Wawrzyniak

June 8, 2026

Architecture of large projects in bioinformatics

I: The challenge of navigating ModelDB repositories

ModelDB purpose and its lack of standarization

- **ModelDB function:** Querying ModelDB for existing neurocomputational models of specific circuits/phenomena (e.g., brain structures, oscillogenesis).
- **ModelDB purpose:** Exploring these models, validating them against own experimental data, comparing model parametrization.
- **ModelDB obstacle:** Model repositories lack structural and format standardization.
 - Code is distributed across many fragmented files.
 - Values and assignments are deeply nested and non-trivial.
 - Navigating the repository is a time-consuming and difficult.
- **Our goal:** Providing a proof-of-concept to navigate and assess projects built on the **NEURON** engine.

The NEURON execution flow

`.hoc` Files

- Mainly integrate other files formats.
- Setup the NEURON environment.
- Define cells, synapses, and relations.
- Initialize networks by evoking mechanisms.

`.mod` Files

- Raw mathematical descriptions (ODEs/PDEs).
- Compiled into low-level C code.
- Translated implicitly during execution.

Execution is not a standard sequential dataflow.

Parameters are assigned in HOC, handed to specific MOD instances, and compiled implicitly.

Practical example - unit deduction

How is the parameter `gA_soma` assigned?

MC_def.hoc

```
// Variable assignment
gA_soma = 10e-3

// Later in the file...
soma {
    insert kamt
    gbar_kamt = gA_soma
}
```



kAmt.mod

```
// Mathematical mechanism
PARAMETER {
    gbar = 0.0002 (mho/cm2)
}
```

Unit deduction: `kamt` maps to `kAmt.mod` → `gbar` has units (mho/cm2) → `gbar_kamt` inherits this → `gA_soma` must be (mho/cm2).

Practical example - structure heterogeneity



Example repository structures analyzed during proof-of-concept validation: Files can be nested in subfolders; documentation (README) can exist in any arbitrary file format; newer repositories frequently include Python scripts; and multiple extensions (*.oc, *.tem, *.hoc) all provide HOC language formatting.

II: Proposed solution

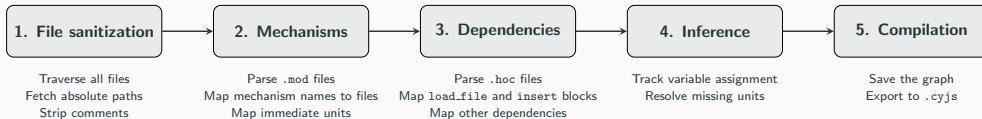
Dependency & unit extraction

We built a static parser to automatically trace these complex relationships in file dependencies and unit deduction, keeping parameters highly interpretable. However, we also note that this parser was meant to follow human-in-the-loop (HITL) approach. No parser can out-think a neuroscientist looking directly at the NEURON code.

Core functionalities:

- **NEURON project parsing:** Scans entire repositories to build a dependency tree.
- **Unit inference:** A multi-attempt algorithm to deduce parameter units across the files.
- **Variable indexing:** Maps all variables declared and used within each file.
- **Graph representation:** Exports structural maps into standard .cyjs format.
- **GUI:** Presents structural maps, indexed variables and inferred units in easily navigable Dash-based GUI. Node connections and assigned units are modifiable in the GUI.

Parser analysis pipeline



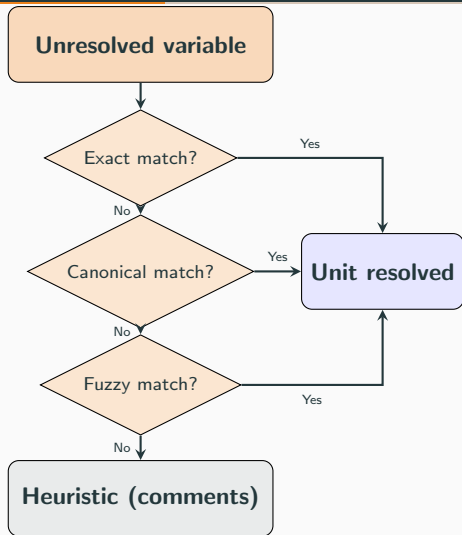
The unit interference waterfall

Step 1: Equivalence pools

We track assignments ($a = b$) to link variables. Once such pool is constructed, if we infer a single variable unit, all others must have the same unit.

Step 2: Waterfall inference

Once pools are constructed, we attempt to infer each variable's unit and resort to assigning the top-match for the whole pool.



Resulting data structure

Our pipeline transforms unstructured folders into structured JSON nodes containing :

```
{
  "nodes": [ {
    "data": {
      "id": "GC_Stim.hoc",
      "type": "hoc",
      "variables": [ {
        "name": "dur",
        "unit": "ms",
        "resolution_method": "exact",
        "resolution_file": "OdorInput.mod"
      } ] }, } ]
  "edges": [ {
    "data": {
      "source": "GC_Stim.hoc",
      "target": "GC_def.hoc",
      "relation": "loads",
      "id": "GC_Stim.hoc_loads_GC_def.hoc"
    } } ]
}
```

We built a GUI for the resulting `.cyjs` file, which presents file dependencies and inferred units.

- **Graph-code view:** Graph and raw source code views to increase tool ergonomics.
- **Graph topological modes:** Dynamic rendering of the graph via hierarchical, force-directed, or circular layout.
- **Orphan node remapping:** Interactive capability to manually insert and wire missing edges for isolated files.
- **Reactive parameter table:** Interactive tabular viewer detailing the exact variables, equivalence pools, and units of each selected node.

III: Conclusions

Conclusions

- **Underestimated heterogeneity:** we significantly underestimated how deeply unstandardized ModelDB projects are. Even common, unofficial structural conventions are rarely implemented across codebases.
- **Execution flow:** mapping the execution flow for individual executable simulation paths proved to be too complex due to NEURON's non-sequential runtime architecture.
- **Semantics:** implementing LLM-driven parameter interpretation would be of huge value. Underlying physiological variables are conceptually standard but their script names are entirely arbitrary.
- **Interpreter:** fully resolving dynamic parameter states at runtime would require writing a full-fledged NEURON language interpreter, which exceeded the scope of this static analysis project.

OBGAMMA

From raw files to interactive graph